



Politechnika Warszawska

**Wydział Matematyki
i Nauk Informatycznych**

Reprezentacja Wiedzy

Procesy działań złożonych

Dokumentacja techniczna

Zespół projektowy nr 4:

Wojciech Jasiński (koordynator)

Krzysztof Gryboś

Jarosław Bieniek

Filip Filipkowski

Sebastian Rydz

Jakub Pietrzak

Sandra Adamiec

Warszawa, 11 czerwca 2026

Spis treści

1	Wprowadzenie	2
2	Architektura	2
3	Języki wejściowe — składnia plikowa	3
4	Przebieg rozwiązywania	3
5	Struktury danych	5
6	Algorytmy	5
6.1	Generacja przestrzeni stanów Σ	6
6.2	Stany początkowe Σ_0 i walidacja modelu (M2)	6
6.3	Res akcji prostej	6
6.4	Wykrywanie konfliktów	6
6.5	Dekompozycje	7
6.6	Res akcji złożonej	7
6.7	Drzewo wykonań i ewaluacja kwerendy	7
6.8	Parsowanie, render grafu i pamięć podręczna	8
7	Realizacja warunków Z1–Z7	8
8	Przykłady z obliczeniami	8
8.1	Prosty efekt — inercja i minimalizacja	8
8.2	Rosyjska ruletka — niedeterminizm przez releases	9
8.3	Konflikt w akcji złożonej — dekompozycje	9
8.4	Ramifikacja — skutek uboczny przez always	9
8.5	Niewykonalność — impossible i kwerenda executable	10
9	Obsługa błędnego wejścia	10
10	Testy	10
10.1	Testy jednostkowe	11
10.2	Testy integracyjne	12
10.3	Testy regresyjne	15
10.4	Uruchamianie i powtarzalność	15

1 Wprowadzenie

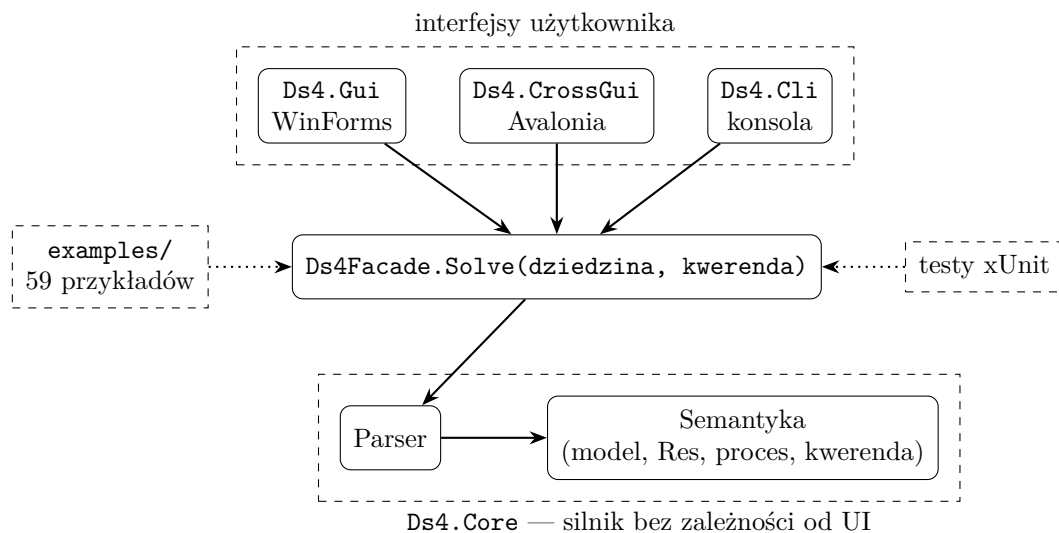
Aplikacja jest solverem dla klasy *DS4* systemów dynamicznych. Na wejściu przyjmuje tekstowy opis dziedziny (w języku akcji) oraz jedną kwerendę (w języku kwerend), buduje w pamięci model dziedziny i odpowiada **TAK** lub **NIE** na pytania Q1 (wykonywalność procesu) i Q2 (osiągnięcie celu), w wariantach *zawsze* i *kiedykolwiek*. Oprócz odpowiedzi program pokazuje krótkie uzasadnienie oraz pełne drzewo wykonań procesu (trace).

Definicje obu języków oraz ich semantyka znajdują się w *dokumentacji teoretycznej*; niniejszy dokument ich nie powtarza — opisuje wyłącznie budowę programu, przyjmowaną składnię plikową, przebieg obliczeń i testy. Tam, gdzie mowa o pojęciach formalnych (Σ , Σ_0 , Res, New, konflikty, dekompozycje), podajemy odsyłacze do odpowiednich sekcji dokumentacji teoretycznej (numeracja według jej wersji 2).

Implementacja: C# 12 / .NET 8. Silnik nie korzysta z żadnych bibliotek zewnętrznych; interfejsy graficzne używają WinForms (Windows) i Avalonii (wieloplatformowo). Testy — xUnit.

2 Architektura

System składa się z jednego silnika (*Ds4.Core*) i trzech wymiennych interfejsów. Całość spina jedna fasada.



- **Interfejsy** (WinForms / Avalonia / CLI) — tylko pola tekstowe, lista wbudowanych przykładów i przycisk „Oblicz”. Trzy frontendy korzystają z tej samej logiki.
- **Ds4Facade** — jedyne API używane przez interfejsy. Przyjmuje dwa napisy (dziedzina, kwerenda), zwraca odpowiedź TAK/NIE, uzasadnienie, rozmiary $|\Sigma|$ i $|\Sigma_0|$, tekstowy trace oraz opis grafu wykonań w formacie Graphviz DOT (*TraceGraphDot*). Wyniki na czterech poziomach trafiają do pamięci podręcznej (statystyki: *CacheStats*).
- **Ds4.Core** — czysty silnik: parser zamienia tekst na struktury danych, moduł semantyki buduje model i ewaluje kwerendę. Wymiana frontendu nie zmienia ani jednej linii silnika.
- **examples/** — 59 par plików *.domain* + *.query*, ładowane do listy w GUI i CLI.
- **Testy** — xUnit; testują fasadę i moduły semantyki, w tym wszystkie przykłady względem plików *.expected*.

3 Języki wejściowe — składnia plikowa

Parser przyjmuje zapis ASCII konstrukcji zdefiniowanych w dokumentacji teoretycznej (sekcje 2.4 i 3.11). Poniższa tabela zestawia notację teoretyczną z zapisem plikowym.

Konstrukcja	Notacja teoretyczna	Zapis w pliku
efekt akcji	$A \textbf{causes } \alpha \textbf{ if } \pi$	shoot causes not alive if loaded
zwolnienie inercji	$A \textbf{releases } f \textbf{ if } \pi$	spin releases loaded if true
niewykonalność	$\textbf{impossible } A \textbf{ if } \pi$	impossible load if loaded
ograniczenie	$\textbf{always } \alpha$	always p implies q
stan początkowy	$\textbf{initially } \alpha$	initially not p and not q
fluent nieinercyjny	$\textbf{noninertial } f$	noninertial f
zdanie wartościujące	$\alpha \textbf{ after } \mathbb{A}_1; \dots; \mathbb{A}_n$	p after a; {b,c}
wariant obserwowalny	$\textbf{observable } \alpha \textbf{ after } \dots$	observable p after a

Formuły zapisuje się wyłącznie operatorami słownymi: **not** ($\neg$), **and** ($\wedge$), **or** ($\vee$), **implies** ($\rightarrow$), **iff** ($\leftrightarrow$) oraz stałymi **true/false**; pusty proces zapisuje się jako **epsilon** (ε). Symbole (**!**, **&**, **|**, **->**, **<->**) nie są akceptowane — parser zgłasza wtedy błąd. Komentarze zaczynają się od **#**. Deklaracje **fluents ...** i **actions ...** są opcjonalne — parser sam zbiera symbole występujące w zdaniach.

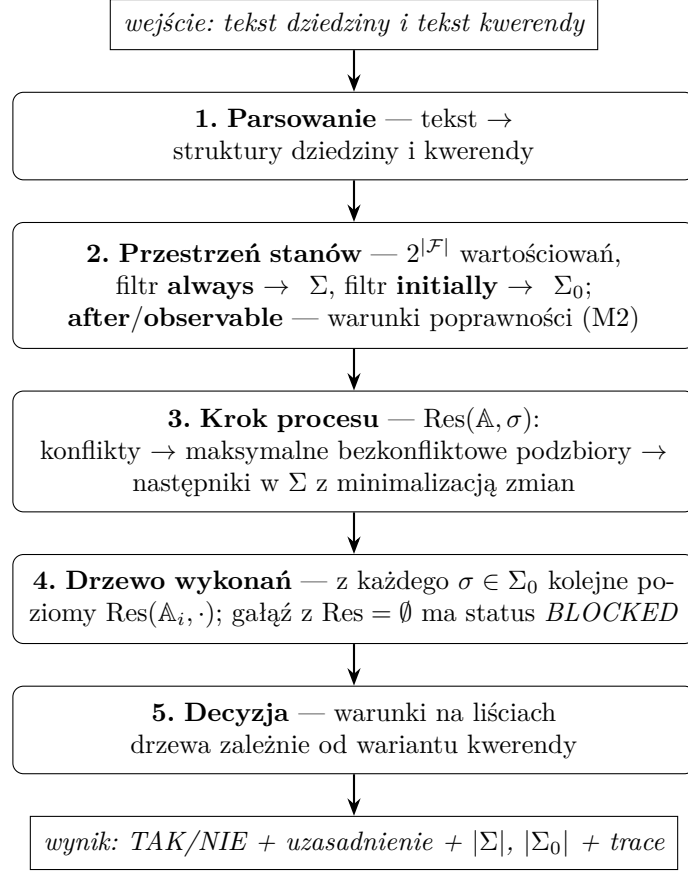
Zgodnie z dokumentacją teoretyczną **impossible** i **initially** są skrótami (odpowiednio: $A \textbf{causes } \perp \textbf{ if } \pi$ oraz $\alpha \textbf{ after } \varepsilon$) — parser rozpoznaje je jako osobne słowa kluczowe, ale semantycznie traktuje tak samo jak rozwinięcia.

Kwerendy. Plik kwerendy zawiera jedno zdanie. Akcję złożoną zapisuje się w nawiasach klamrowych (**{a,b}**; nawiasy można pominąć dla akcji jednoelementowej), proces — jako ciąg akcji złożonych rozdzielonych średnikami. Cztery warianty pytań:

Pytanie	Wariant	Zapis w pliku
Q1: wykonywalność	zawsze	necessary executable after a; {b,c}
	kiedykolwiek	possibly executable after a; {b,c}
Q2: cel γ	zawsze	necessary not alive after spin; shoot
	kiedykolwiek	possibly not alive after spin; shoot

4 Przebieg rozwiązywania

Obliczenie przebiega w pięciu krokach. Program realizuje wprost definicje z dokumentacji teoretycznej — dla małych dziedzin (kilka–kilkanaście fluentów) pełne przeszukiwanie $2^{|\mathcal{F}|}$ stanów jest szybkie, a wierność definicjom była ważniejsza niż optymalizacje.



Krótko o każdym kroku (w nawiasach sekcje dokumentacji teoretycznej z definicjami):

1. **Parsowanie.** Z tekstu powstaje lista fluentów, akcji, reguł **causes** / **releases** / **impossible**, ograniczeń **always** / **initially** oraz proces i cel kwerendy.
2. **Przestrzeń stanów.** Generujemy wszystkie wartościowania fluentów i odsiewamy niezgodne z **always** — zostaje Σ (sekcja 2.5.1). Z Σ wybieramy stany zgodne z **initially** — zostaje Σ_0 (sekcja 2.5.2), czyli zbiór możliwych stanów początkowych przy opisie częściowym. Zbiór ten wyznaczają *wyłącznie* zdania **initially**. Zdania **after** oraz **observable after** to warunki poprawności modelu (M2): sprawdzamy je globalnie nad całym Σ_0 i *nie* usuwamy z niego pojedynczych stanów. Zdanie α **after** P wymaga, by z *każdego* stanu początkowego *każda* pełna ścieżka procesu P kończyła się stanem spełniającym α ; wariant **observable** — by tak kończyła się *przynajmniej jedna* ścieżka z *pewnego* stanu początkowego. Niespełnienie któregoś z warunków oznacza dziedzinę sprzeczną (brak modelu, $\Sigma_0 = \emptyset$).
3. **Krok procesu.** Dla akcji prostej: jeśli aktywna jest reguła **impossible**, następników brak; w przeciwnym razie następnikami są stany z Σ spełniające aktywne efekty i minimalizujące zbiór zmian New (sekcje 2.5.3–2.5.6) — tak realizujemy inercję (Z1) i skutki uboczne ograniczeń (Z6). Fluent zwolniony regułą **releases** zawsze wnosi swój literał do New, dzięki czemu oba jego wartościowania są nieporównywalne i oba przeżywają minimalizację (niedeterminizm, Z3). Dla akcji złożonej: wykrywamy konflikty par akcji w stanie σ (sekcja 3.3), wyznaczamy maksymalne bezkonfliktowe podzbiory — dekompozycje (sekcja 3.4) — i jako $\text{Res}(A, \sigma)$ bierzemy sumę wyników po dekompozycjach (sekcja 3.5).
4. **Drzewo wykonań.** Z każdego stanu początkowego rozwijamy drzewo: dzieci korzenia to $\text{Res}(A_1, \sigma)$, wnuki — $\text{Res}(A_2, \cdot)$ itd. (sekcja 3.8–3.9). Gałąź ucięta przed końcem procesu jest *zablokowana*; gałąź pełnej długości kończy się *liściem końcowym*.

5. **Decyzja** (sekcja 3.12). Kwantyfikator po stanach początkowych jest zawsze uniwersalny: przy opisie częściowym prawdziwy stan początkowy nie jest znany, więc odpowiedź musi uwzględniać każdy stan zgodny z **initially**. Warunki (we wszystkich przypadkach wymagamy $\Sigma_0 \neq \emptyset$):

- **possibly executable** — z *każdego* stanu początkowego istnieje liść końcowy;
- **possibly γ** — z *każdego* stanu początkowego istnieje liść końcowy spełniający γ ;
- **necessary executable** — brak gałęzi zablokowanych i istnieją liście końcowe;
- **necessary γ** — jak wyżej oraz *wszystkie* liście końcowe spełniają γ .

5 Struktury danych

Silnik Ds4.Core odwzorowuje pojęcia z dokumentacji teoretycznej na zwarte, niezmiennie struktury C#. Poniżej zestawiono najważniejsze z nich wraz z plikiem źródłowym w katalogu `src/Ds4.Core/`.

- **Stan σ** — `SortedSet<string>` prawdziwych fluentów; równość przez `SetEquals`, skrót przez `HashCode` (bez rozróżniania wielkości liter). Plik `Model/State.cs`.
- **Dziedzina** — zbiory `HashSet` fluentów, akcji i fluentów **noninertial**; listy rekordów `CauseRule`, `ReleaseRule`, `AlwaysConstraint`, `InitiallyConstraint` oraz `AfterAssertion` (cel, proces, znacznik *observable*). Plik `Model/Domain.cs`.
- **Formuła** — drzewo `IFormula` (`And`, `Or`, `Not`, ...) z metodami `Evaluate(State)` i `ToDnf()`. Katalog `Formula/`.
- **Składnik DNF** — `DnfTerm`, czyli mapa `Dictionary<string, bool>` (`fluent  $\rightarrow$  polaryzacja`) z metodą `HasComplementWith`. Plik `Formula/IFormula.cs`.
- **Model przejść** — `TransitionModel` przechowuje Σ oraz Σ_0 jako `IReadOnlyList<State>`. Plik `Semantics/TransitionModel.cs`.
- **Drzewo wykonań** — `ExecutionNode` (pola `State`, `Depth`, `IncomingStep`, `Blocked`, `Children`) oraz `ExecutionTree` z listą korzeni i `ProcessLength`. Plik `Semantics/ExecutionTrace.cs`.
- **Kwerenda** — `CompositeAction` i `Ds4Process`; rekord `Ds4Query` (kwantyfikator, rodzaj, proces, cel). Plik `Query/QueryModel.cs`.
- **Pamięć podręczna** — cztery słowniki `ConcurrentDictionary` (formuły, modele dziedzin, kwerendy, wyniki) oraz `CacheStats`. Plik `Api/Ds4Facade.cs`.

Stan przechowuje *tylko* fluenty prawdziwe, co czyni równość i haszowanie niezależnymi od kolejności oraz pozwala używać stanów jako kluczy w zbiorach (`HashSet<State>`) przy deduplikacji wyników Res. Formuły efektów są sprowadzane do DNF, ponieważ wykrywanie konfliktów i komplementarności literałów operuje na składnikach (zob. §6).

6 Algorytmy

Poniżej zebrano kluczowe procedury silnika w postaci pseudokodu, każdą wraz z realizującą ją metodą. Procedury wprost realizują definicje z dokumentacji teoretycznej; dla małych dziedzin pełne przeszukiwanie $2^{|\mathcal{F}|}$ stanów jest tanie, więc wybrano wierność definicjom zamiast optymalizacji.

6.1 Generacja przestrzeni stanów Σ

Metoda `StateGenerator.GenerateAll` wylicza wszystkie $2^{|\mathcal{F}|}$ wartościowania (po bitach maski), a `ModelBuilder.Build` odsiewa te, które łamią choć jedno ograniczenie **always**.

```
funkcja GenerujSigma(dziedzina D):
   $\mathcal{F}$  := posortowane fluenty D
   $\Sigma$  := []
  dla maski m od 0 do  $2^{|\mathcal{F}|} - 1$ :
     $\sigma$  := {  $\mathcal{F}[i]$  : i-ty bit m ustawiony }
    jeli dla wszystkich (always  $\alpha$ )  $\in D$  zachodzi  $\sigma \models \alpha$ :
      dodaj  $\sigma$  do  $\Sigma$ 
  zwr  $\Sigma$ 
```

6.2 Stany początkowe Σ_0 i walidacja modelu (M2)

Zbiór Σ_0 wyznaczają *wyłącznie* zdania **initially**. Zdania **after** oraz **observable after** nie są filtrem Σ_0 , lecz globalnym warunkiem poprawności (M2) sprawdzanym nad całym Σ_0 ; ich niespełnienie czyni dziedzinę sprzeczną ($\Sigma_0 := \emptyset$). Realizuje to `ModelBuilder.Build`.

```
funkcja GenerujSigma0(D,  $\Sigma$ ):
   $\Sigma_0$  := {  $\sigma \in \Sigma$  :  $\sigma \models \alpha$  dla wszystkich (initially  $\alpha$ )  $\in D$  }
  dla kadego zdania (obserwowalne?,  $\alpha$  after P)  $\in D$ :           // warunek M2
    jeli obserwowalne: // istnieje stan pocztkowy ze śladem konczącym się  $\alpha$ 
      ok := istnieje  $\sigma \in \Sigma_0$ : jakiś liśc StanyKoncowe( $\sigma$ , P)  $\models \alpha$ 
    inaczej: // z kadego stanu kadzy ślad konczy się  $\alpha$ 
      ok := dla kadego  $\sigma \in \Sigma_0$ : kadzy liśc StanyKoncowe( $\sigma$ , P)  $\models \alpha$ 
    jeli nie ok: zwr  $\Sigma_0 := \emptyset$  // dziedzina spreczna
  zwr  $\Sigma_0$ 
```

6.3 Res akcji prostej

`SimpleActionEngine.Res`: jeśli aktywna jest reguła **impossible**, wynik jest pusty; inaczej kandydatami są stany z Σ spełniające aktywne efekty, spośród których zostają te minimalne względem zbioru zmian `New`.

```
funkcja Res(A,  $\sigma$ ):
  jeli aktywna regua (impossible A if  $\pi$ ),  $\sigma \models \pi$ : zwr  $\emptyset$ 
  E := aktywne efekty A w  $\sigma$ ; R := aktywne zwolnienia A w  $\sigma$ 
  Res0 := {  $\sigma' \in \Sigma$  :  $\sigma' \models \alpha$  dla wszystkich  $\alpha \in E$  }
  dla kadego  $\sigma' \in \text{Res}_0$ :
    New[ $\sigma'$ ] := { literal l :  $\sigma' \models l$  oraz
      ( l zmienia fluent inercyjny:  $\sigma(f) \neq \sigma'(f)$  ) lub ( fluent l  $\in R$  ) }
  zwr {  $\sigma' \in \text{Res}_0$  : nie istnieje  $\sigma''$  z New[ $\sigma''$ ]  $\subsetneq$  New[ $\sigma'$ ] }
```

6.4 Wykrywanie konfliktów

Zgodnie z dokumentacją teoretyczną (sekcja *Konflikty*) dwie akcje proste kolidują w σ *tylko* w dwóch przypadkach. Samo operowanie na wspólnym fluencie konfliktu *nie* powoduje. Realizuje to `ConflictDetector.AreInConflict`.

```
funkcja Koliduja(A, B,  $\sigma$ ):
   $E_A, E_B$  := składniki DNF aktywnych efektów;  $R_A, R_B$  := aktywne zwolnienia
  // Przypadek 1 (causes vs causes): literal komplementarne
  jeli istnieją  $C_A \in E_A$ ,  $C_B \in E_B$  ze wspólnym fluentem f, f  $\in C_A$ ,  $\neg f \in C_B$  (lub
    odwrotnie):
```

```

    zwr prawda
// Przypadek 2 (causes vs releases): efekt jednej dotyka fluentu zwalnianego przez
// druga
jeli ( $E_A$  wymienia jakis  $f \in R_B$ ) lub ( $E_B$  wymienia jakis  $f \in R_A$ ):
    zwr prawda
zwr fałsz

```

6.5 Dekompozycje

`DecompositionGenerator.Generate`: spośród akcji indywidualnie wykonalnych ($\text{Res} \neq \emptyset$) wyznacza *maksymalne* podzbiory bezkonfliktowe — maksymalne zbiory niezależne w grafie konfliktu.

```

funkcja Dekompozycje( $\mathbb{A}$ ,  $\sigma$ ):
    V := { A ∈  $\mathbb{A}$  : Res(A,  $\sigma$ ) ≠ ∅ } // indywidualnie wykonalne
    K := { {A,B} : A,B ∈ V, Koliduja(A, B,  $\sigma$ ) } // krawedzie konfliktu
    kandydaci := { X ⊆ V : zaden konflikt wewnatrz X }
    zwr { X ∈ kandydaci : X nie jest podzbiorem wiekszego kandydata }

```

6.6 Res akcji złożonej

`CompositeActionEngine.Res`: dla każdej dekompozycji łączymy jej efekty i zwolnienia, liczymy lokalnie Res jak dla akcji prostej, a wynik akcji złożonej to *suma* po dekompozycjach. Minimalizacja New jest *lokalna* w obrębie dekompozycji — świadoma decyzja projektowa względem globalnego warunku (M3) języka AC. Konflikt daje więc niedeterminizm: może zająć wynik dowolnej z dekompozycji.

```

funkcja Res( $\mathbb{A}$ ,  $\sigma$ ):
     $\Delta$  := Dekompozycje( $\mathbb{A}$ ,  $\sigma$ )
    jeli  $\Delta = \emptyset$ : zwr ∅
    wynik := ∅
    dla kadej  $\delta \in \Delta$ :
        E :=  $\bigcup_{A \in \delta}$  efekty(A,  $\sigma$ ); R :=  $\bigcup_{A \in \delta}$  zwolnienia(A,  $\sigma$ )
        wynik := wynik ∪ ResZRegu(E, R,  $\sigma$ ) // jak w Res prostej, z minimalizacja New
    zwr wynik

```

6.7 Drzewo wykonań i ewaluacja kwerendy

`ProcessEvaluator.BuildTree` rozwija drzewo od każdego $\sigma \in \Sigma_0$; gałąź z pustym Res otrzymuje status *Blocked*. `QueryEvaluator.Evaluate` odczytuje odpowiedź z liści. Kwantyfikator po Σ_0 jest zawsze uniwersalny (przy opisie częściowym stan początkowy nie jest znany), więc *possibly* znaczy „dla każdego σ_0 istnieje gałąź”, a nie „istnieje σ_0 ”.

```

funkcja Ewaluuj(kwerenda q):
    drzewo := BuildTree( $\Sigma_0$ , q.proces) // korzen na poziomie 0 dla kazdego  $\sigma_0$ 
    jeli  $\Sigma_0 = \emptyset$ : zwr NIE
    zalenie od q:
        possibly executable: dla kazdego korzenia istnieje lisc na poziomie n
        possibly  $\gamma$ : dla kazdego korzenia istnieje lisc na poziomie n z  $\models \gamma$ 
        necessary executable: brak galezi zablockowanych i istnieja liscie na poziomie n
        necessary  $\gamma$ : jw. oraz wszystkie liscie na poziomie n  $\models \gamma$ 

```


6.8 Parsowanie, render grafu i pamięć podręczna

Parser formuł (`FormulaParser`) to rekursywny zejściowy parser z priorytetami `iff < implies < or < and < unarne`; **DomainParser** czyta dziedzinę wierszami. Sparsowane formuły trafiają do `ConcurrentDictionary` (po normalizacji tekstu). **Render grafu** (`TraceGraphRenderer.RenderDot`) zamienia drzewo wykonania na opis Graphviz DOT (kolorywanie: stan początkowy, końcowy, zablokowany), z limitem `maxNodes` chroniącym przed eksplozją. **Fasada** (`Ds4Facade.Solve`) spina całość i zapamiętuje wyniki na czterech poziomach (formuły, modele dziedzin, sparsowane kwerendy, wyniki `Solve`); statystyki podaje `CacheStats`. Brak zależności zewnętrznych i źródeł losowości czyni każdy przebieg powtarzalnym.

7 Realizacja warunków Z1–Z7

Treść projektu definiuje klasę *DS4* siedmioma warunkami. Poniższa tabela wskazuje, który mechanizm programu realizuje każdy z nich.

Warunek	Realizacja w programie
Z1. Prawo inercji	minimalizacja zbioru zmian New w Res: fluent nieobjęty efektem zachowuje wartość
Z2. Pełna informacja	jawna enumeracja wszystkich stanów ($2^{ \mathcal{F} }$) i wszystkich reguł dziedziny — nic nie jest domyślne
Z3. Niedeterminizm działań	releases (oba wartościowania zwolnionego fluentu przeżywają minimalizację) oraz suma wyników po dekompozycjach akcji złożonej
Z4. Warunek wstępny i wynikowy	reguły <i>A causes α if π</i> ; gdy π nie zachodzi, reguła jest nieaktywna (efekt pusty)
Z5. Niewykonalność	impossible A if π — Res = $\emptyset$, gałąź drzewa dostaje status BLOCKED
Z6. Warunki integralności	filtr always na Σ ; następniki wybierane wyłącznie z Σ , więc ograniczenia wymuszają skutki uboczne (ramifikacja)
Z7. Opis częściowy	Σ_0 jako zbiór <i>wszystkich</i> stanów zgodnych z initially ; after/observable to warunki poprawności (M2), nie filtry na Σ_0 ; uniwersalny kwantyfikator po Σ_0 w obu rodzajach kwerend

8 Przykłady z obliczeniami

Pięć przykładów z wbudowanego zestawu; każdy pokazuje inny mechanizm. Zapisy dziedzin i kwerend są dosłownie tym, co przyjmuje program.

8.1 Prosty efekt — inercja i minimalizacja

```
initially not p
make causes p if true
```

Kwerenda: `necessary p after make`

- $\mathcal{F} = \{p\}$, brak **always**, więc $\Sigma = \{\{p\}, \{\neg p\}\}$; **initially not p** daje $\Sigma_0 = \{\{\neg p\}\}$.
- Res(*make*, $\neg p$): aktywny efekt *p*; kandydaci spełniający *p*: tylko $\{p\}$, New = $\{p\}$ — wynik $\{\{p\}\}$.
- Jedyny liść końcowy spełnia *p*, gałęzi zablokowanych brak. **Odpowiedź: TAK.**

8.2 Rosyjska ruletka — niedeterminizm przez releases

```

fluents loaded, alive
actions spin, shoot, load
initially alive
spin releases loaded if true
shoot causes not alive if loaded
impossible load if loaded
load causes loaded if not loaded

```

Kwerendy: possibly not alive after spin; shoot oraz necessary not alive after spin; shoot

- $|\Sigma| = 4$; **initially alive** ogranicza tylko *alive*, więc Σ_0 ma dwa stany: $\{l, a\}$ i $\{\neg l, a\}$ (opis częściowy).
- $\text{Res}(\text{spin}, \sigma)$: brak efektów **causes**, zwolniony *loaded* — oba wartościowania *loaded* przeżywają minimalizację, więc z każdego stanu początkowego dostajemy $\{l, a\}$ i $\{\neg l, a\}$.
- $\text{Res}(\text{shoot}, \cdot)$: z $\{l, a\}$ aktywny efekt $\neg \text{alive}$ daje $\{l, \neg a\}$; z $\{\neg l, a\}$ żadna reguła nie jest aktywna — inercja zostawia $\{\neg l, a\}$.
- Z każdego z dwóch stanów początkowych istnieje więc ścieżka kończąca się $\{l, \neg a\} \models \neg \text{alive}$ — ale wśród liści jest też $\{\neg l, a\}$, więc nie wszystkie ścieżki kończą się celem. **Odpowiedzi: possibly — TAK, necessary — NIE.**

8.3 Konflikt w akcji złożonej — dekompozycje

```

initially not p
makeP causes p if true
clearP causes not p if true

```

Kwerenda: necessary p after {makeP, clearP}

- $\Sigma = \{\{p\}, \{\neg p\}\}$, $\Sigma_0 = \{\{\neg p\}\}$.
- W stanie $\neg p$ akcje *makeP* i *clearP* wymuszają literały przeciwne (*p* i $\neg p$) — konflikt. Maksymalne bezkonfliktowe podzbiory: $\{\text{makeP}\}$ i $\{\text{clearP}\}$.
- $\text{Res}(\{\text{makeP}, \text{clearP}\}, \neg p) = \text{Res}(\text{makeP}, \neg p) \cup \text{Res}(\text{clearP}, \neg p) = \{\{p\}, \{\neg p\}\}$.
- Liść $\{\neg p\}$ nie spełnia *p*. **Odpowiedź: NIE.**

8.4 Ramifikacja — skutek uboczny przez always

```

always p implies q
initially not p and not q
makeP causes p if true

```

Kwerenda: necessary q after makeP

- Z czterech wartościowań ograniczenie **always p implies q** odrzuca $\{p, \neg q\}$, więc $|\Sigma| = 3$; $\Sigma_0 = \{\{\neg p, \neg q\}\}$.
- $\text{Res}(\text{makeP}, \{\neg p, \neg q\})$: aktywny efekt *p*; jedynym stanem w Σ spełniającym *p* jest $\{p, q\}$ — fluent *q* zmienia się jako skutek uboczny, bo stan $\{p, \neg q\}$ nie należy do Σ .
- Jedyny liść spełnia *q*. **Odpowiedź: TAK.**

8.5 Niewykonalność — impossible i kwerenda executable

```
fluents loaded
actions load
initially loaded
impossible load if loaded
load causes loaded if not loaded
```

Kwerenda: possibly executable after load

- $\Sigma = \{\{l\}, \{\neg l\}\}$, $\Sigma_0 = \{\{l\}\}$.
- $\text{Res}(\text{load}, \{l\})$: aktywna jest reguła **impossible**, więc następników brak — jedyna gałąź drzewa jest zablokowana już na pierwszym kroku.
- Ze stanu początkowego nie istnieje żadna pełna ścieżka. **Odpowiedź: NIE.**

9 Obsługa błędnego wejścia

Fasada nie rzuca wyjątków do interfejsu — każdy problem z wejściem kończy się czytelnym komunikatem zamiast odpowiedzi:

- **błąd składni** w dziedzinie lub kwerendzie — zwracany jest komunikat parsera wskazujący niepoprawne zdanie;
- **sprzeczne ograniczenia always** — żaden stan ich nie spełnia ($\Sigma = \emptyset$); program zgłasza: „Model jest pusty: ograniczenia always są sprzeczne”;
- **sprzeczny opis początkowy** — $\Sigma_0 = \emptyset$; wszystkie kwerendy mają wtedy odpowiedź NIE, a w wyniku widać $|\Sigma_0| = 0$, co wskazuje przyczynę;
- **bardzo duże drzewo wykonań** — trace jest ucinany po około 60 tys. znaków z jawną adnotacją, żeby nie blokować GUI; odpowiedź i uzasadnienie są liczone zawsze na pełnym drzewie.

10 Testy

Poprawność silnika weryfikujemy zautomatyzowanym zestawem testów (framework xUnit, projekt `Ds4.Tests`). Zestaw liczy **114 metod testowych**, które po rozwinięciu testów parametryzowanych (`[Theory]`) dają **247 przypadków testowych** — wszystkie zielone, czas wykonania poniżej 0,3 s. Testy działają w trybie wsadowym (bez interfejsu graficznego) wprost na czystym silniku `Ds4.Core`, w większości przez tę samą fasadę `Ds4Facade.Solve`, której używają wszystkie frontendy. Silnik nie ma zależności zewnętrznych ani żadnego źródła losowości, więc każdy przebieg jest w pełni *powtarzalny*.

Przyjęliśmy trójwarstwową strategię testowania, od najdrobniejszych jednostek do całych scenariuszy:

- **testy jednostkowe** — sprawdzają pojedyncze moduły semantyki w izolacji (parser, formuły, model, Res, dekompozycje, drzewo wykonań);
- **testy integracyjne** — sprawdzają cały tor obliczeń „od tekstu do TAK/NIE” na zestawie wbudowanych przykładów;
- **testy regresyjne** — utrwalają poprawione interpretacje semantyczne, aby raz naprawiony błąd nie wrócił.

10.1 Testy jednostkowe

Każdy moduł silnika ma własny plik testowy. Testy budują minimalną dziedzinę z napisu (pomocnik `TestData.BuildModel/Solve`) i sprawdzają jeden aspekt zachowania, dzięki czemu ewentualna regresja od razu wskazuje winny moduł.

Plík testowy	Co sprawdza	Testów
<code>ParserTests</code>	składnia dziedziny, kwerendy, procesu i formuł; odrzucanie błędnych zdań	9
<code>FormulaTests</code>	wartościowanie formuł, priorytety operatorów, postać DNF, prawa de Morgana	7
<code>ModelTests</code>	równość stanów (niezależna od kolejności i wielkości liter), wnioskowanie fluentów i akcji	4
<code>StateAndModelBuilderTests</code>	generacja Σ ($2^{ \mathcal{F} }$ wartościowań, filtr always) i Σ_0 (initially); wykrywanie sprzecznych ograniczeń	6
<code>SimpleActionEngineTests</code>	Res akcji prostej: causes , releases (niedeterminizm), impossible , inercja, ramifikacje, akcja pusta	8
<code>CompositeActionTests</code>	wykrywanie konfliktów, generacja dekompozycji, wykonanie akcji złożonej	8
<code>ProcessAndQueryTests</code>	ewaluacja procesu, drzewo wykonań, cztery warianty kwerend (<i>possibly/necessary</i> $\times$ wykonalność/cel)	9
<code>WordOnlySyntaxTests</code>	wymóg operatorów słownych (not/and/...) i odrzucanie symboli	4
<code>LanguageCoverageTests</code>	pokrycie wszystkich konstrukcji języka akcji i kwerend	10
<code>CacheAndGraphTests</code>	pamięć podręczna parsera/wyników oraz render grafu wykonań (DOT)	6

Przykłady wybranych testów jednostkowych (każdy buduje minimalną dziedzinę i sprawdza jeden aspekt):

Przykład 1: Parser dziedziny — rozpoznanie wszystkich typów zdań

```
var domain = DomainParser.Parse("""
    fluents loaded, alive
    actions spin, shoot
    initially alive
    always alive or not loaded
    spin releases loaded if true
    shoot causes not alive if loaded
    impossible shoot if not loaded
    noninertial loaded
    """);
Assert.Single(domain.ReleaseRules);
Assert.Single(domain.CauseRules);
Assert.Single(domain.ImpossibleRules);
```

Przykład 2: Reguła releases daje niedeterminizm w funkcji Res

```
var result = engine.Res("spin", model.Sigma0.Single()); // spin releases loaded
Assert.Equal(2, result.Count);
Assert.Contains(result, s => s.IsTrue("loaded"));
Assert.Contains(result, s => !s.IsTrue("loaded"));
```

Przykład 3: Konflikt rozkłada akcję złożoną na jednoelementowe dekompozycje

```
// make_p causes p ; make_not_p causes not p (literaly komplementarne)
var decompositions = env.Decompositions.Generate(
    new CompositeAction(new[] { "make_p", "make_not_p" }, sigma);
Assert.Equal(2, decompositions.Count);
```

```
Assert.Contains(decompositions, d => d.Count == 1 && d.Contains("make_p"));
Assert.Contains(decompositions, d => d.Count == 1 && d.Contains("make_not_p"));
```

10.2 Testy integracyjne

Najszerszą warstwę stanowi **59 wbudowanych przykładów** w katalogu `app/examples/`. Każdy przykład to trójka plików o wspólnej nazwie: *id.domain* (dziedzina), *id.query* (kwerenda) oraz *id.expected* (oczekiwana odpowiedź: pojedyncze słowo TAK albo NIE). Test parametryzowany ([Theory] + [MemberData], źródło `TestData.ExampleFiles`) ładuje każdą trójkę, rozwiązuje ją przez `Ds4Facade.Solve` i porównuje wynik z plikiem `.expected`. Te same przykłady zasilają listę w GUI i CLI, więc test pełni podwójną rolę: weryfikuje silnik *oraz* gwarantuje, że każdy przykład pokazywany użytkownikowi daje deklarowaną odpowiedź.

Zestaw przykładów dzieli się na **34 z odpowiedzią TAK** i **25 z NIE**, a nazwa każdego koduje jego pochodzenie: pliki rozstrzygnięć mają schemat `{tak|nie}_{tex|extra|bugfix|regression}_nn_opis`, a scenariusze fabularne — `scenario_nn_opis`:

Rodzina	Pochodzenie	TAK	NIE
tex	scenariusze z wykładu i dokumentacji teoretycznej (rosyjska ruletka, producent i konsumenci, dwa przełączniki, dwóch robotników)	13	7
extra	przykłady uzupełniające pokrycie konstrukcji języka	10	10
bugfix	utrwalenie błędów wykrytych w rozwoju (np. warunkowe causes)	1	1
regression	naprawione błędy interpretacji semantyki (częściowy stan początkowy)	0	1
scenario	rozbudowane scenariusze fabularne (m.in. pięciu filozofów, robot dostawczy, ramifikacje, rzut monetą)	10	6
Razem		34	25

Spójności samego zestawu pilnują dodatkowe testy: każda dziedzina musi mieć parę `.query` + `.expected`, a pliki `.expected` mogą zawierać wyłącznie TAK lub NIE.

Test parametryzowany ([Theory] + [MemberData]) wczytuje każdą trójkę plików, wywołuje `Ds4Facade.Solve` i porównuje wynik z plikiem `.expected`. Poniżej reprezentatywne przykłady *wejścia* z rodzin `tex` i `scenario` — każdy odpowiada plikom `.domain/.query/.expected` w katalogu `examples/` — wraz z oczekiwaną odpowiedzią i krótkim uzasadnieniem.

Przykład 4: YSP — problem strzelania (rodzina tex)

```
initially alive
load causes loaded if true
impossible load if loaded
shoot causes not loaded if true
shoot causes not alive if loaded
spin releases loaded if true
not loaded after shoot
# kwerenda:
necessary not alive after load; spin; shoot
```

Odpowiedź: NIE. Dlaczego: spin zwalnia loaded, więc na jednej gałęzi broń jest rozładowana i shoot nie zabija. Istnieje gałąź kończąca się alive, zatem śmierć nie jest konieczna. (Zdanie not loaded after shoot to warunek poprawności M2, spełniony przez model.)

Przykład 5: Producent–konsument (rodzina tex)

```
initially not empty and hasA and hasB
```

```

put causes not empty if true
getA causes empty and hasA if not empty
getB causes empty and hasB if not empty
consumeA causes not hasA if hasA
consumeB causes not hasB if hasB
# kwerenda:
possibly empty after {put,getA,getB,consumeA,consumeB}; {consumeA,consumeB}

```

Odpowiedź: TAK. Dlaczego: kroki to akcje złożone z kolidujących producentów i konsumentów; pewna dekompozycja prowadzi do stanu `empty`, więc dla każdego stanu początkowego istnieje ścieżka realizująca cel.

Przykład 6: Dwa przełączniki — ramifikacja i zwolnienie (rodzina tex)

```

always light iff (sw1 or sw2)
always light implies alarm
noninertial alarm
toggle1 causes sw1 if not sw1
toggle1 causes not sw1 if sw1
toggle2 causes sw2 if not sw2
toggle2 causes not sw2 if sw2
on causes light if true
flicker releases light if true
initially not sw1 and not sw2
# kwerenda:
necessary alarm after {toggle1,toggle2}; {on,flicker}

```

Odpowiedź: NIE. Dlaczego: flicker zwalnia light; istnieje gałąź, na której światło gaśnie (a spójność `light iff (sw1 or sw2)` wymusza wtedy także wyłączenie przełączników), więc nieinercyjny alarm nie jest już wymuszany i może być fałszywy — alarm nie zachodzi koniecznie.

Przykład 7: Robot dostawczy — proces wieloetapowy (rodzina scenario)

```

initially not has_key and not door_open and not package_delivered
find_key causes has_key if true
open_door causes door_open if has_key
impossible open_door if not has_key
deliver causes package_delivered if door_open
impossible deliver if not door_open
# kwerenda:
necessary package_delivered after find_key; open_door; deliver

```

Odpowiedź: TAK. Dlaczego: każda akcja spełnia swój warunek wstępny w kolejności `find_key` → `open_door` → `deliver`; proces jest deterministyczny, a jedyny stan końcowy spełnia `package_delivered`.

Przykład 8: Rzut monetą — niedeterminizm przez releases (rodzina scenario)

```

initially heads
toss releases heads if true
# kwerenda:
possibly not heads after toss

```

Odpowiedź: TAK. Dlaczego: toss zwalnia heads, więc Res zawiera stan z `not heads`; z jednego stanu początkowego istnieje gałąź realizująca cel.

Przykład 9: Efekt alternatywny wsparty akcją — akcja złożona bez konfliktu (rodzina scenario)

```

initially not f and not g
choose causes f or g if true
force_f causes f if true
# kwerenda:
necessary f after {choose, force_f}

```

Odpowiedź: TAK. Dlaczego: `choose` i `force_f` nie kolidują (brak literalów komplementarnych mimo wspólnego `f`), więc tworzą jedną dekompozycję; `force_f` wymusza `f` niezależnie od wyboru `choose`, zatem `f` zachodzi w każdym stanie końcowym.

Przykład 10: Niezależne akcje współbieżne — brak konfliktu (rodzina scenario)

```
initially not bowl_lifted and not door_open
lift_bowl causes bowl_lifted if true
open_door causes door_open if true
# kwerenda:
necessary bowl_lifted and door_open after {lift_bowl, open_door}
```

Odpowiedź: TAK. Dlaczego: obie akcje wpływają na rozłączne fluenty (`bowl_lifted` i `door_open`), więc nie kolidują — jedyna dekompozycja `{lift_bowl, open_door}` wykonuje obie naraz i jedyny stan końcowy spełnia oba cele.

Przykład 11: Indyk — ramifikacja przez `always` (rodzina scenario)

```
initially not loaded and walking and alive
always walking implies alive
load causes loaded if true
shoot causes not loaded if true
shoot causes not alive if loaded
# kwerenda:
necessary not walking after load; shoot
```

Odpowiedź: TAK. Dlaczego: `load` nabija broń, `shoot` uśmierca (`not alive`); z ograniczenia `always walking implies alive` wynika, że stan bez życia musi być też stanem bez chodzenia (ramifikacja), więc `not walking` zachodzi w każdym stanie końcowym.

Przykład 12: Karta i drzwi — akcja zablokowana (rodzina scenario)

```
initially not open and not has_card
insert_card causes open if true
impossible insert_card if not has_card
# kwerenda:
possibly executable after insert_card
```

Odpowiedź: NIE. Dlaczego: w stanie początkowym brakuje `has_card`, więc reguła `impossible insert_card if not has_card` blokuje akcję — Res jest puste i nie istnieje pełna ścieżka.

Przykład 13: Malowanie — więź integralności wymusza zmianę (rodzina scenario)

```
initially yellowA and not yellowB
always not (yellowA and yellowB)
paintB causes yellowB if true
# kwerenda:
necessary yellowB and not yellowA after paintB
```

Odpowiedź: TAK. Dlaczego: `paintB` wymusza `yellowB`; ponieważ `always not (yellowA and yellowB)` zakazuje obu naraz, jako skutek uboczny (ramifikacja) `yellowA` zmienia się na fałsz — jedyny stan końcowy spełnia `yellowB and not yellowA`.

Przykład 14: Test alarmu — `releases` znosi konieczność (rodzina scenario)

```
initially alarm
test_alarm releases alarm if true
# kwerenda:
necessary alarm after test_alarm
```

Odpowiedź: NIE. Dlaczego: `test_alarm` zwalnia `alarm` spod inercji, więc Res zawiera także stan z `not alarm`; istnieje gałąź bez alarmu, zatem `alarm` nie zachodzi koniecznie.

Niezależny test sprawdza ponadto, że `Ds4Facade.Solve` zwraca dla każdego przykładu poprawny opis grafu wykonań w formacie Graphviz DOT (`TraceGraphDot`).

10.3 Testy regresyjne

Część testów powstała w odpowiedzi na konkretne, naprawione błędy interpretacji semantyki — utrwalają one decyzje opisane w *dokumentacji teoretycznej* (wersja 2) i chronią przed ich cofnięciem:

- Σ_0 **nie jest zawężane** przez zdania **after** ani **observable**. Zdanie **after** jest warunkiem poprawności modelu (M2): jego naruszenie unieważnia całą dziedzinę ($\Sigma_0 = \emptyset$), a nie usuwa pojedynczego stanu (`After_Assertion_Is_Validity_Condition_Not_A_Filter...`). Zdanie **observable** jest egzystencjalne i również nie usuwa stanów początkowych, nawet jeśli z niektórych celu nie da się osiągnąć (`Observable_After_Does_Not_Trim_Initial_States...`).
- *possibly* **kwantyfikuje uniwersalnie po** Σ_0 (dla *każdego* stanu początkowego istnieje ścieżka do celu), a nie egzystencjalnie. Plik `PossiblySemanticsTests` (7 testów) sprawdza m.in., że brak ścieżki z jednego stanu daje odpowiedź NIE, oraz że dla jednoelementowego Σ_0 wynik pokrywa się ze starą, egzystencjalną semantyką.
- **warunkowe causes** ($A \text{ causes } q \text{ if } p$) działają tylko na gałęziach, gdzie warunek p zachodzi (`ConditionalCauseRegressionTests`, 6 testów).

Przykład 15: Regresja: warunkowe causes na gałęzi, gdzie warunek p zachodzi

```
// dziedzina: initially not q ; action causes q if p
var start = model.Sigma0.Single(s => s.IsTrue("p") && !s.IsTrue("q"));
var result = engine.Res("action", start);
Assert.Single(result);
Assert.True(result.Single().IsTrue("q")); // efekt wymuszony bo p
```

10.4 Uruchamianie i powtarzalność

Cały zestaw uruchamia polecenie `dotnet test` w katalogu `app/` (na Windows także skrypt `RUN_TESTS.bat`). Wymagane jest jedynie .NET 8; brak zależności zewnętrznych sprawia, że wynik jest identyczny na każdej maszynie. Przypisanie testów do członków zespołu znajduje się w osobnym dokumencie „Wkład osobowy”.